

[USER] ps

DESCRIPTION : Affiche un instantané des processus en cours d'exécution  
 POURQUOI : Voir quels processus tournent, leur état, leur consommation CPU/RAM.  
 CONTEXTE : Monitoring, diagnostic de charge, identification de processus suspects.  
 OPTIONS :

|              |                                                          |
|--------------|----------------------------------------------------------|
| aux          | Tous les processus de tous les utilisateurs (format BSD) |
| -ef          | Tous les processus (format System V)                     |
| -u USER      | Processus d'un utilisateur spécifique                    |
| -p PID       | Processus d'un PID spécifique                            |
| -C NOM       | Processus par nom de commande                            |
| f / --forest | Arbre de processus (affiche la hiérarchie)               |
| -o FORMAT    | Colonnes personnalisées                                  |
| --sort=COL   | Trie par colonne                                         |
| -L           | Affiche les threads                                      |
| -T           | Affiche les threads avec ID de thread                    |

COLONNES CLÉS :

|         |                                                   |
|---------|---------------------------------------------------|
| PID     | Process ID – identifiant unique                   |
| PPID    | Parent PID – processus parent                     |
| %CPU    | Pourcentage CPU utilisé                           |
| %MEM    | Pourcentage mémoire utilisée                      |
| VSZ     | Mémoire virtuelle totale (Ko)                     |
| RSS     | Mémoire physique réelle (Ko)                      |
| STAT    | État (R=Running, S=Sleeping, Z=Zombie, T=Stopped) |
| TTY     | Terminal associé                                  |
| START   | Heure de démarrage                                |
| COMMAND | Commande complète                                 |

EXEMPLES :

```
$ ps aux
$ ps aux | grep nginx
$ ps -ef --forest
$ ps -o pid,ppid,%cpu,%mem,cmd aux | sort -k3 -rn | head -10
```

[USER] top

DESCRIPTION : Moniteur de processus en temps réel (tableau de bord interactif)  
 POURQUOI : Voir en direct la charge CPU, l'utilisation mémoire, les processus les plus gourmands. Outil de diagnostic de performance numéro 1.  
 CONTEXTE : Monitoring de performance, diagnostic de charge serveur, dépannage.  
 OPTIONS :

|         |                                            |
|---------|--------------------------------------------|
| -d N    | Intervalle de rafraîchissement en secondes |
| -n N    | Nombre de rafraîchissements puis quitte    |
| -u USER | Filtre par utilisateur                     |
| -p PID  | Surveille un PID spécifique                |
| -b      | Mode batch (non interactif, pour scripts)  |
| -H      | Affiche les threads individuellement       |
| -c      | Affiche la commande complète               |

RACCOURCIS INTERACTIFS :

|   |                                     |
|---|-------------------------------------|
| q | Quitter                             |
| k | Tuer un processus (entre le PID)    |
| r | Modifier la priorité (renice)       |
| M | Trier par utilisation mémoire       |
| P | Trier par utilisation CPU           |
| T | Trier par temps CPU cumulé          |
| l | Afficher chaque cœur CPU séparément |
| H | Basculer affichage threads          |
| f | Gérer les colonnes affichées        |
| z | Activer/désactiver les couleurs     |
| W | Sauvegarder la configuration        |

EXEMPLES :

```
$ top
$ top -u www-data
$ top -b -n 1 | head -20
$ top -b -d 5 > /tmp/top_monitoring.txt
```

[USER] htop (si installé)

DESCRIPTION : Version améliorée de top avec interface colorée et navigation  
 POURQUOI : Bien plus ergonomique que top – navigation avec les flèches, kill direct, arbre de processus visuel.  
 CONTEXTE : Monitoring interactif, administration quotidienne.  
 OPTIONS :  
   -u USER Filtre par utilisateur  
   -p PID[,PID...] Surveillance des PIDs spécifiques  
   -s COL Tri initial  
   -d N Intervalle de rafraîchissement (dixièmes de seconde)  
   --tree Vue arborescente au démarrage  
 EXEMPLES :  
   \$ htop  
   \$ htop -u root

```
[USER] kill
```

DESCRIPTION : Envoie un signal à un processus par son PID  
 POURQUOI : Terminer, mettre en pause, ou recharger des processus.  
 CONTEXTE : Arrêt de processus bloqués, rechargement de services, gestion de jobs.  
 SIGNAUX COURANTS :  
   -1 / -HUP Rechargement de configuration (Hang Up)  
   -2 / -INT Interruption (équivalent Ctrl+C)  
   -9 / -KILL Terminaison forcée – IRRÉVERSIBLE, pas de nettoyage  
   -15 / -TERM Terminaison propre (défaut) – laisse le temps de nettoyer  
   -17 / -STOP Suspension du processus (pause)  
   -18 / -CONT Reprise d'un processus suspendu  
   -19 / -TSTP Suspension (équivalent Ctrl+Z)  
   -l Liste tous les signaux disponibles  
 EXEMPLES :  
   \$ kill 1234  
   \$ kill -9 1234  
   \$ kill -HUP \$(pgrep nginx)  
   \$ kill -l

```
[USER] killall
```

DESCRIPTION : Envoie un signal à tous les processus portant un nom donné  
 POURQUOI : Tuer tous les processus d'une application sans chercher les PIDs.  
 CONTEXTE : Arrêt de services, nettoyage de processus orphelins.  
 OPTIONS :  
   -9 Signal SIGKILL (force)  
   -u USER Limite aux processus de cet utilisateur  
   -i Mode interactif (demande confirmation)  
   -v Mode verbeux  
   -r REGEX Nom en expression régulière  
   -s SIGNAL Signal à envoyer  
   -w Attend que les processus se terminent  
 EXEMPLES :  
   \$ killall firefox  
   \$ killall -9 zombie\_process  
   \$ killall -u www-data php-fpm

```
[USER] pkill
```

DESCRIPTION : Envoie un signal aux processus selon des critères variés (nom, user...)  
 POURQUOI : Plus flexible que killall – peut filtrer par utilisateur, terminal, etc.  
 CONTEXTE : Gestion fine de processus, administration.  
 OPTIONS :  
   -u USER Processus de cet utilisateur  
   -g PGRP Processus de ce groupe  
   -t TTY Processus de ce terminal  
   -P PPID Enfants de ce processus parent  
   -n Tue seulement le processus le plus récent  
   -o Tue seulement le processus le plus ancien  
   -x Correspondance exacte du nom  
   -f Correspondance sur la ligne de commande complète  
   -l Affiche le nom du processus tué  
   -e Affiche le nom et le PID  
 EXEMPLES :  
   \$ pkill firefox  
   \$ pkill -u alice  
   \$ pkill -f "python mon\_script.py"

```
$ pkill -9 -t pts/1
```

```
[USER] pgrep
```

```
DESCRIPTION : Recherche des processus par nom et affiche leurs PIDs
POURQUOI : Récupérer les PIDs d'un processus pour les passer à d'autres commandes.
CONTEXTE : Scripts, monitoring, kill ciblé.
OPTIONS :
  -u USER      Filtre par utilisateur
  -l           Affiche le nom avec le PID
  -a           Affiche la commande complète
  -x           Correspondance exacte
  -f           Correspondance sur la commande complète
  -n           Processus le plus récent uniquement
  -o           Processus le plus ancien uniquement
  -c           Affiche seulement le nombre
EXEMPLES :
$ pgrep nginx
$ pgrep -u root -l
$ kill -HUP $(pgrep sshd)
```

```
[USER] pidof
```

```
DESCRIPTION : Trouve le PID d'un programme par son nom exact
POURQUOI : Récupérer le PID d'un daemon pour des scripts.
CONTEXTE : Scripts d'init, vérification qu'un service tourne.
OPTIONS :
  -s           Retourne seulement un PID (le premier trouvé)
  -x           Inclut les scripts shell
  -o PID       Omet ce PID des résultats
EXEMPLES :
$ pidof nginx
$ pidof -s sshd
$ PID=$(pidof -s apache2)
```

```
[USER] pstree
```

```
DESCRIPTION : Affiche les processus sous forme d'arbre hiérarchique
POURQUOI : Visualiser les relations parent-enfant entre processus.
CONTEXTE : Diagnostic de processus orphelins, compréhension de l'arborescence.
OPTIONS :
  -a           Affiche les arguments des commandes
  -p           Affiche les PIDs
  -u           Affiche les propriétaires
  -h           Met en évidence le processus courant
  -H PID       Met en évidence ce PID
  -n           Trie par PID (pas par nom)
  -s           Affiche les parents du processus sélectionné
EXEMPLES :
$ pstree
$ pstree -aup
$ pstree -p | grep nginx
```

```
[USER] nice
```

```
DESCRIPTION : Lance une commande avec une priorité d'ordonnancement modifiée
POURQUOI : Réduire la priorité d'une tâche intensive pour ne pas perturber les autres processus (ex: compilation, sauvegarde).
CONTEXTE : Tâches de fond gourmandes, maintenance sans impacter les services.
OPTIONS :
  -n N         Définit la valeur nice (-20 le plus prioritaire, +19 le moins)
NOTES :
  Valeur nice positive (1 à 19) Réduit la priorité – tout utilisateur peut le faire
  Valeur nice négative (-1 à -20) Augmente la priorité – SUDO requis
EXEMPLES :
$ nice -n 19 tar czf archive.tar.gz /home/
$ sudo nice -n -10 processus_critique
```

```
[USER] renice
```

DESCRIPTION : Modifie la priorité d'un processus déjà en cours d'exécution  
 POURQUOI : Ajuster dynamiquement la priorité sans redémarrer le processus.  
 CONTEXTE : Optimisation dynamique des ressources, dépannage de charge.  
 OPTIONS :  
   -n N Nouvelle valeur nice  
   -p PID Cible un PID  
   -u USER Cible tous les processus d'un utilisateur  
   -g PGRP Cible un groupe de processus  
 NOTES :  
   Augmenter la priorité (nice négatif) nécessite SUDO  
 EXEMPLES :  
   \$ renice -n 10 -p 1234  
   \$ sudo renice -n -5 -p 1234  
   \$ sudo renice -n 19 -u backup\_user

[USER] jobs

DESCRIPTION : Liste les jobs (processus) en cours dans le shell courant  
 POURQUOI : Voir les processus mis en arrière-plan avec & ou Ctrl+Z.  
 CONTEXTE : Gestion des jobs shell, reprise de processus suspendus.  
 OPTIONS :  
   -l Affiche les PIDs avec les jobs  
   -p Affiche seulement les PIDs  
   -r Affiche seulement les jobs en cours (Running)  
   -s Affiche seulement les jobs suspendus (Stopped)  
 EXEMPLES :  
   \$ sleep 100 &  
   \$ jobs  
   \$ jobs -l

[USER] bg

DESCRIPTION : Reprend un job suspendu en arrière-plan  
 POURQUOI : Reprendre un processus mis en pause (Ctrl+Z) sans le ramener au premier plan.  
 CONTEXTE : Gestion des jobs shell.  
 EXEMPLES :  
   \$ bg  
   \$ bg %2

[USER] fg

DESCRIPTION : Ramène un job d'arrière-plan au premier plan  
 POURQUOI : Reprendre le contrôle d'un processus en arrière-plan.  
 CONTEXTE : Gestion des jobs shell.  
 EXEMPLES :  
   \$ fg  
   \$ fg %1  
   \$ fg %2

[USER] nohup

DESCRIPTION : Lance une commande immunisée au signal HUP (déconnexion)  
 POURQUOI : Garder un processus en vie après déconnexion SSH – fondamental pour les tâches longues lancées depuis SSH.  
 CONTEXTE : Tâches longues via SSH, scripts de maintenance, déploiements.  
 OPTIONS :  
   Redirige stdout vers nohup.out par défaut  
 EXEMPLES :  
   \$ nohup ./long\_script.sh &  
   \$ nohup python3 serveur.py > /var/log/serveur.log 2>&1 &  
   \$ nohup rsync -av /data/ /backup/ &

[USER] wait

DESCRIPTION : Attend la fin d'un ou plusieurs processus en arrière-plan  
 POURQUOI : Synchroniser des scripts parallèles – attendre que tous les jobs d'arrière-plan soient terminés avant de continuer.  
 CONTEXTE : Scripts parallèles, synchronisation de jobs.  
 EXEMPLES :  
   \$ long\_job1 &  
   wait

```
$ long_job2 &
$ wait
$ echo "Tous les jobs sont terminés"
$ wait $PID1 $PID2
```

[USER] flock

DESCRIPTION : Gère des verrous de fichiers pour éviter l'exécution simultanée  
POURQUOI : Garantir qu'un seul script s'exécute à la fois (protection des ressources partagées, scripts cron).  
CONTEXTE : Scripts cron, gestion de ressources partagées, exclusion mutuelle.  
OPTIONS :

|         |                                      |
|---------|--------------------------------------|
| -e / -x | Verrou exclusif (défaut)             |
| -s      | Verrou partagé                       |
| -n      | Non bloquant (échoue si verrou pris) |
| -w N    | Attend N secondes maximum            |
| -u      | Libère le verrou                     |
| -c CMD  | Exécute CMD sous le verrou           |

EXEMPLES :

```
$ flock -n /tmp/mon_script.lock ./mon_script.sh
$ flock /var/lock/backup.lock -c "rsync -av /data/ /backup/"
$ ( flock -n 9 || exit 1; commande ) 9>/tmp/lock
```

[USER] timeout

DESCRIPTION : Lance une commande avec une limite de temps d'exécution  
POURQUOI : Éviter qu'une commande reste bloquée indéfiniment.  
CONTEXTE : Scripts d'automatisation, requêtes réseau, tests.  
OPTIONS :

|                   |                                                             |
|-------------------|-------------------------------------------------------------|
| DURÉE             | Durée avec unité : s=secondes, m=minutes, h=heures, d=jours |
| -s SIGNAL         | Signal à envoyer à l'expiration (défaut: TERM)              |
| -k DURÉE          | Délai avant SIGKILL si SIGNAL ignoré                        |
| --preserve-status | Retourne le code de sortie de la commande                   |
| --foreground      | Permet les signaux en mode interactif                       |

EXEMPLES :

```
$ timeout 30 wget https://example.com
$ timeout 5m ./script_long.sh
$ timeout -s KILL 10 commande_bloquante
```

[USER] time

DESCRIPTION : Mesure le temps d'exécution d'une commande  
POURQUOI : Benchmarking une commande, mesurer les performances.  
CONTEXTE : Optimisation, benchmarking, profiling de scripts.  
SORTIE :

|      |                                 |
|------|---------------------------------|
| real | Temps total écoulé (wall clock) |
| user | Temps CPU en mode utilisateur   |
| sys  | Temps CPU en mode noyau         |

EXEMPLES :

```
$ time find / -name "*.conf" 2>/dev/null
$ time python3 script.py
$ time sort -n fichier_large.txt > /dev/null
```

[USER] watch

DESCRIPTION : Exécute une commande périodiquement et affiche les résultats  
POURQUOI : Surveiller en temps réel une valeur ou un état qui change.  
CONTEXTE : Monitoring, surveillance de logs, suivi de fichiers.  
OPTIONS :

|         |                                                  |
|---------|--------------------------------------------------|
| -n N    | Intervalle en secondes (défaut: 2)               |
| -d      | Met en évidence les différences entre exécutions |
| -t      | Supprime l'en-tête                               |
| -e      | Arrête si la commande retourne une erreur        |
| -g      | Arrête quand la sortie change                    |
| --color | Préserve les couleurs ANSI                       |
| -x      | Utilise exec au lieu de sh -c (plus précis)      |

EXEMPLES :

```
$ watch -n 1 'netstat -an | grep ESTABLISHED | wc -l'
$ watch -d df -h
$ watch -n 5 'tail -n 5 /var/log/nginx/access.log'
```

## [USER] strace

DESCRIPTION : Trace les appels système effectués par un processus  
POURQUOI : Déboguer un programme qui se comporte de manière inattendue – voir exactement ce qu'il fait au niveau OS (fichiers ouverts, réseau...).

CONTEXTE : Débogage avancé, analyse de malware, reverse engineering, optimisation.

OPTIONS :

- p PID S'attache à un processus existant
- e trace=APPELS Filtre les appels système (ex: open,read,write)
- e trace=file Appels liés aux fichiers
- e trace=network Appels réseau
- e trace=signal Signaux
- o FICHIER Écrit la trace dans un fichier
- f Suit les processus enfants (fork)
- t Ajoute les timestamps
- T Affiche le temps passé dans chaque appel
- c Affiche des statistiques de résumé
- s N Longueur maximale des chaînes affichées

EXEMPLES :

```
$ strace ls /tmp
$ strace -p 1234 -e trace=file
$ strace -f -o /tmp/trace.log ./programme
$ strace -c df -h
```

## [USER] ltrace

DESCRIPTION : Trace les appels aux bibliothèques partagées d'un processus  
POURQUOI : Voir quelles fonctions de bibliothèques (libc, libssl...) sont appelées.

CONTEXTE : Débogage, analyse de programme, optimisation.

OPTIONS :

- p PID S'attache à un processus existant
- l BIBLIO Filtre une bibliothèque spécifique
- f Suit les forks
- c Résumé statistique
- o FICHIER Sortie vers un fichier

EXEMPLES :

```
$ ltrace ls
$ ltrace -l libssl.so.1 ./monprogramme
```

## [USER] lsof

DESCRIPTION : Liste tous les fichiers ouverts par les processus (LiSt Open Files)  
POURQUOI : Sous Linux, "tout est fichier" – sockets réseau, pipes, devices sont aussi des fichiers. lsof révèle toutes les ressources utilisées.

CONTEXTE : Diagnostic réseau, identifier qui utilise un fichier/port, sécurité.

OPTIONS :

- p PID Fichiers ouverts par ce PID
- u USER Fichiers ouverts par cet utilisateur
- c NOM Fichiers ouverts par ce programme
- i Connexions réseau (Internet files)
- i :PORT Connexions sur ce port
- i TCP Connexions TCP uniquement
- n Pas de résolution DNS (plus rapide)
- P Pas de résolution de port
- +D RÉPERTOIRE Fichiers ouverts dans ce répertoire
- t Affiche seulement les PIDs (pour scripts)
- r N Mode répétition toutes les N secondes

EXEMPLES :

```
$ lsof -i :80
$ lsof -u www-data
$ lsof /var/log/nginx/access.log
$ lsof -i TCP -n -P | grep LISTEN
$ lsof +D /var/www/html
```

## [USER] fuser

DESCRIPTION : Identifie les processus utilisant des fichiers ou sockets  
POURQUOI : Savoir quel processus bloque un fichier ou utilise un port.

CONTEXTE : Débogage de montages occupés, identification de processus sur un port.

OPTIONS :

- k Tue les processus utilisant la ressource

```

-i          Mode interactif avant de tuer
-u          Affiche le nom de l'utilisateur
-m          Le fichier est sur ce système de fichiers
-n tcp PORT Processus sur ce port TCP
-n udp PORT Processus sur ce port UDP
-v          Mode verbeux

```

EXEMPLES :

```

$ fuser 80/tcp
$ fuser -ku /mnt/disque
$ fuser -v /var/log/messages

```

```
[USER] vmstat
```

DESCRIPTION : Rapport sur les statistiques mémoire, processus, I/O, CPU et swap

POURQUOI : Vue synthétique des performances système – identifier les goulots d'étranglement (mémoire, swap, I/O, CPU).

CONTEXTE : Monitoring de performance, diagnostic de saturation.

OPTIONS :

```

INTERVALLE      Rafraîchissement en secondes
COUNT          Nombre de rapports
-a              Affiche la mémoire active/inactive
-d              Statistiques disque
-s              Table de statistiques mémoire
-S m            Affiche en mégaoctets
-t              Ajoute les timestamps
-w              Format large

```

COLONNES CLÉ :

```

r              Processus en attente de CPU (run queue)
b              Processus bloqués (en attente d'I/O)
swpd           Mémoire swap utilisée
free           Mémoire libre
si/so          Swap in/out (ko/s) – si >0, problème mémoire!
bi/bo          Blocs disque in/out par seconde
us/sy/id/wa    CPU: user/system/idle/wait I/O

```

EXEMPLES :

```

$ vmstat 1 10
$ vmstat -s
$ vmstat -d 2

```

```
[USER] iostat
```

DESCRIPTION : Statistiques d'utilisation CPU et d'entrées/sorties disque

POURQUOI : Identifier les disques saturés, mesurer les performances I/O.

CONTEXTE : Diagnostic de performance disque, monitoring de bases de données.

OPTIONS :

```

-x              Statistiques étendues (les plus utiles)
-d              Seulement les statistiques disque
-c              Seulement les statistiques CPU
-h              Format lisible
-k              En Ko/s
-m              En Mo/s

```

INTERVALLE COUNT Rafraîchissement

EXEMPLES :

```

$ iostat -x 2 5
$ iostat -d -h 1

```

```
[USER] free
```

DESCRIPTION : Affiche l'utilisation de la mémoire RAM et swap

POURQUOI : Vérifier la mémoire disponible, voir si le swap est utilisé.

CONTEXTE : Monitoring de mémoire, diagnostic de saturation, scripts de surveillance.

OPTIONS :

```

-h              Format lisible (Ko, Mo, Go)
-m              En mégaoctets
-g              En gigaoctets
-k              En kilooctets (défaut)
-t              Affiche le total RAM + swap
-s N            Rafraîchissement toutes les N secondes
-c N            Nombre de rafraîchissements
-w              Format large (colonnes séparées buffer/cache)

```

EXEMPLES :

```

$ free -h

```

```
$ free -h -s 2
$ watch -n 1 free -h
```

```
[USER] uptime
```

DESCRIPTION : Affiche depuis combien de temps le système tourne et la charge moyenne  
POURQUOI : Vérifier la disponibilité du serveur et sa charge (load average).  
CONTEXTE : Monitoring, vérification post-reboot, scripts de santé.  
OPTIONS :  
-p Format lisible ("up 2 days, 3 hours")  
-s Date et heure de démarrage du système  
LOAD AVERAGE : Les 3 valeurs = charge moyenne sur 1, 5, 15 minutes  
Valeur < nombre de cœurs CPU = charge acceptable  
EXEMPLES :  
\$ uptime  
\$ uptime -p  
\$ uptime -s

```
[USER] pmap
```

DESCRIPTION : Affiche la carte mémoire d'un processus (mappings mémoire)  
POURQUOI : Analyser comment un processus utilise la mémoire, trouver les fuites.  
CONTEXTE : Débogage mémoire, profiling, optimisation d'applications.  
OPTIONS :  
-x Format étendu  
-d Format device  
-q Pas d'en-tête ni de pied de page  
EXEMPLES :  
\$ pmap -x 1234  
\$ pmap \$(pgrep java) | tail -1

```
[USER] taskset
```

DESCRIPTION : Définit ou récupère l'affinité CPU d'un processus  
POURQUOI : Forcer un processus à s'exécuter sur des cœurs CPU spécifiques – optimisation NUMA, isolation de performances.  
CONTEXTE : Optimisation HPC, isolation de processus critiques, benchmarking.  
OPTIONS :  
-c LISTE Spécifie les CPUs par numéro (ex: 0,1,4-7)  
-p PID Applique à un processus existant  
EXEMPLES :  
\$ taskset -c 0,1 ./programme  
\$ taskset -c 2-5 -p 1234  
\$ taskset -p 1234

```
[USER] chrt
```

DESCRIPTION : Définit la politique d'ordonnancement temps-réel d'un processus  
POURQUOI : Donner une priorité temps-réel à un processus critique.  
CONTEXTE : Applications temps-réel, audio/vidéo, systèmes embarqués.  
OPTIONS :  
-f Politique FIFO (temps-réel)  
-r Politique Round-Robin (temps-réel)  
-b Politique BATCH  
-i Politique IDLE (la plus basse)  
-o Politique OTHER (normale)  
-p PID Processus existant  
PRIORITÉ Niveau de 1 (bas) à 99 (haut)  
EXEMPLES :  
\$ sudo chrt -f 50 ./audio\_programme  
\$ chrt -p 1234

```
[USER] ionice
```

DESCRIPTION : Définit la classe et la priorité d'I/O d'un processus  
POURQUOI : Réduire l'impact I/O d'une tâche de fond sur les autres processus.  
CONTEXTE : Sauvegardes, synchronisation, tâches de maintenance.  
OPTIONS :  
-c CLASSE 0=None, 1=Realtime, 2=Best-effort, 3=Idle  
-n N Niveau de priorité (0-7, 0 le plus prioritaire)



```
-p PID                Processus existant
EXEMPLES              :
$ ionice -c 3 rsync -av /data/ /backup/
$ ionice -c 2 -n 7 tar czf archive.tar.gz /home/
$ ionice -p 1234
```

[USER] nsenter

```
DESCRIPTION : Entre dans les namespaces d'un processus existant
POURQUOI    : Entrer dans les namespaces d'un conteneur pour déboguer de l'intérieur,
              sans shell dans le conteneur.
CONTEXTE    : Débogage de conteneurs, analyse de namespaces Linux.
OPTIONS      :
  -t PID      Processus cible
  -m          Namespace de montage
  -u          Namespace UTS (hostname)
  -i          Namespace IPC
  -n          Namespace réseau
  -p          Namespace PID
  -U          Namespace utilisateur
EXEMPLES      :
$ sudo nsenter -t $(pgrep nginx) -n ip addr
$ sudo nsenter -t 1234 --all bash
```

[USER] unshare

```
DESCRIPTION : Lance un programme dans de nouveaux namespaces Linux
POURQUOI    : Créer des environnements isolés sans conteneur complet – test de
              namespaces, création de sandbox.
CONTEXTE    : Conteneurisation légère, tests d'isolation, sécurité.
OPTIONS      :
  -m          Nouveau namespace de montage
  -u          Nouveau namespace UTS
  -i          Nouveau namespace IPC
  -n          Nouveau namespace réseau
  -p          Nouveau namespace PID
  -U          Nouveau namespace utilisateur
  --fork      Fork avant d'exécuter la commande
  --pid       Nouveau namespace PID avec PID 1 dans le namespace
EXEMPLES      :
$ unshare -n ip addr
$ sudo unshare -m --propagation private bash
```

[USER] pidwait

```
DESCRIPTION : Attend la fin d'un ou plusieurs processus par leur nom
POURQUOI    : Attendre qu'un processus nommé se termine (comme wait mais par nom).
CONTEXTE    : Scripts de synchronisation, attente de daemons.
EXEMPLES      :
$ pidwait rsync
$ pidwait -x mon_processus
```

[USER] skill / snice

```
DESCRIPTION : skill : envoie un signal par critères variés
              snice : modifie la priorité par critères variés
POURQUOI    : Gestion de processus par critères multiples (user, terminal, commande).
CONTEXTE    : Administration, gestion de sessions utilisateurs.
EXEMPLES      :
$ skill -KILL -u alice
$ snice -19 -u processus_important
```

[USER] tload

```
DESCRIPTION : Affiche graphiquement la charge système dans le terminal
POURQUOI    : Visualisation rapide de la charge sous forme de graphique ASCII.
CONTEXTE    : Monitoring léger en terminal.
OPTIONS      :
  INTERVALLE  Rafraîchissement en secondes
  -s N        Définit l'échelle du graphique
```

EXEMPLES :  
\$ tload  
\$ tload 2

[USER] irqtop

DESCRIPTION : Moniteur d'interruptions matérielles en temps réel (comme top pour les IRQ)  
POURQUOI : Diagnostiquer des problèmes matériels via les interruptions.  
CONTEXTE : Diagnostic matériel, débogage de drivers.  
EXEMPLES :  
\$ irqtop  
\$ irqtop -d 2

[USER] slabtop

DESCRIPTION : Affiche les informations sur les slabs de la mémoire noyau en temps réel  
POURQUOI : Diagnostiquer la consommation mémoire du noyau Linux.  
CONTEXTE : Débogage noyau, diagnostic de fuites mémoire kernel.  
OPTIONS :  
-d N Intervalle de rafraîchissement  
-s COL Colonne de tri  
-o Une seule itération  
EXEMPLES :  
\$ sudo slabtop  
\$ sudo slabtop -s c

[USER] choom

DESCRIPTION : Ajuste le score OOM Killer d'un processus  
POURQUOI : Contrôler lequel des processus sera tué en premier quand la mémoire est épuisée (OOM = Out Of Memory).  
CONTEXTE : Serveurs critiques – protéger des processus importants du OOM Killer.  
OPTIONS :  
-n SCORE Score OOM (-1000 à 1000, plus haut = tué en premier)  
-p PID Processus cible  
EXEMPLES :  
\$ sudo choom -n -500 -p \$(pgrep mysql)  
\$ choom -p 1234